

1. Contextualització del projecte

1.1. Introducció

En aquest document es presenta el meu Treball de Fi de Grau (TFG) titulat «**Desenvolupament de funcionalitats per a l'aplicació Localboss**».

Aquest projecte s'emmarca dins del Grau d'Enginyeria Informàtica (GEI) de la Universitat Politècnica de Catalunya (UPC), amb especialització en Enginyeria del Programari.

El TFG s'inscriu en la modalitat B, que implica la col·laboració amb una empresa del sector del desenvolupament de programari. En aquest cas, l'empresa és **Localboss** [\[G1\]](#) [\[R10\]](#), una *start-up* que desenvolupa una aplicació mòbil destinada a ajudar els propietaris de negocis a gestionar les seves ressenyes a **Google Maps** [\[G2\]](#) [\[R11\]](#).

1.2. Descripció detallada de l'empresa

Localboss és una empresa petita que es va crear a finals del 2021 amb un objectiu molt clar: desenvolupar una aplicació mòbil que permeti als propietaris de negocis gestionar les seves ressenyes a Google Maps.

Com que l'empresa és jove i de dimensions reduïdes, no disposa d'oficines i tots els empleats treballen en modalitat de teletreball. Aquesta decisió no només suposa un estalvi econòmic, sinó que també facilita la incorporació de professionals ubicats en diferents punts d'Espanya.

L'equip actual està format per vuit persones: dos fundadors, un treballador fix, dos estudiants en pràctiques i tres professionals *freelance*. Cada membre s'especialitza en components diferents que abasten des de vendes/màrqueting fins a l'àrea tècnica.

L'aplicació que desenvolupa, i que porta el mateix nom, té com a funció ajudar a gestionar les ressenyes de Google Maps.

1.3. Descripció detallada de l'aplicació

L'aplicació de LOCALBOSS permet gestionar de manera segura i centralitzada les ressenyes d'un negoci a Google Maps, aportant informació addicional que Google no mostra de manera nativa.

Entre les seves funcionalitats principals:

- **Nota real:** mostra la puntuació exacta (no arrodonida), i indica quantes ressenyes falten per aconseguir la següent millora de nota (per exemple, de 4,6 a 4,7).
- **Gràfics i estadístiques:** ofereix visualitzacions diàries, mensuals i anuals del *rating*, així com la mitjana de puntuacions i el volum de ressenyes per períodes.
- **Gestió de ressenyes:** permet respondre-les directament i aplicar filtres per data, puntuació o estat de resposta.
- **Plantilles i resposta amb IA:** inclou un sistema de respostes predefinides i un sistema de resposta automàtica mitjançant IA configurable, que genera respostes personalitzades segons el contingut de la ressenya.

1.4. Descripció problema

L'aplicació de Localboss va ser desenvolupada amb Flutter [\[G3\]](#) [\[R9\]](#). Com que cap dels 2 fundadors era expert en aquest framework, van aconseguir crear una aplicació funcional, però amb mancances tant de qualitat de codi com de funcionalitats.

Per aquest motiu, l'empresa va decidir centrar-se en millorar el producte en lloc de prioritzar el creixement inicial. Així, van contractar primer un *freelance* especialista en Flutter i posteriorment un estudiant en pràctiques per donar suport en tasques de millora i refactorització.

Actualment, l'aplicació té un codi més net i estable respecte la primera versió i ha anat creixent progressivament. Per això, el focus actual és el desenvolupament de noves funcionalitats i canvis d'estil més ambiciosos.

1.5. Actors implicats

En aquest projecte intervenen diferents actors i parts interessades:

- **Director del TFG:** Jordi Gil de Bernabé, cofundador de Localboss i responsable de fer tot el backend [\[G4\]](#) de l'aplicació. Supervisa el projecte i proporciona els endpoints necessaris per a la integració amb el frontend [\[G5\]](#).
- **Ponent del TFG:** Carles Farré Tost, professor de la UPC i encarregat de guiar l'autor durant tot el procés del treball.
- **Autor del TFG:** estudiant responsable del desenvolupament, que assumeix rols de desenvolupador frontend, tester i enginyer de software.
- **Equip de Localboss:** col·laboren aportant dissenys visuals, revisant codi i fent proves de les noves funcionalitats.
- **Usuaris:** clients de l'aplicació que utilitzen les noves funcionalitats i proporcionen feedback.
- **Competidors:** altres aplicacions que ofereixen serveis similars de gestió de ressenyes a Google Maps.

9. Especificació

Aquest capítol descriu de manera formal i estructurada l'especificació del sistema desenvolupat, detallant les funcionalitats finals implementades, els requisits funcionals i no funcionals, així com el model conceptual del domini. L'objectiu d'aquest apartat és definir **què fa el sistema i quines condicions ha de complir**, establint una base sòlida i verificable per a les fases posteriors de disseny, implementació i validació.

El projecte s'ha desenvolupat en un context professional real dins l'empresa Localboss, sobre una aplicació mòbil en producció amb usuaris actius. Aquest fet ha condicionat fortament l'especificació, ja que els requisits no només responen a necessitats funcionals, sinó també a criteris de qualitat, estabilitat, seguretat i evolució del producte.

L'especificació inicial del projecte ha evolucionat al llarg del desenvolupament seguint una metodologia àgil. Algunes funcionalitats previstes inicialment han estat substituïdes per altres de major valor per al producte final, com és el cas de la incorporació de l'agent d'intel·ligència artificial **Cintia** en lloc de la funcionalitat Share Review, o la incorporació de **contingut multimèdia a les reviews** en substitució de la gestió de permisos de delegated users.

9.1. Requisits funcionals

Els requisits funcionals es descriuen mitjançant **Històries d'Usuari**, cadascuna amb els seus criteris d'acceptació associats. Aquest enfocament permet una definició clara, entenedora i validable del comportament esperat del sistema.

9.1.1. RF-01: Autenticació mitjançant Delegated Login

Història d'usuari:

Com a usuari delegat, vull poder iniciar sessió mitjançant el meu correu electrònic o número de telèfon, perquè pugui accedir a l'aplicació sense necessitat d'un compte principal.

Criteris d'acceptació

- El sistema ha de permetre introduir un correu electrònic o un número de telèfon.
- El sistema ha d'enviar un codi OTP a l'adreça o número indicat.
- L'usuari ha de poder introduir el codi OTP rebut.
- Si el codi és correcte, l'usuari ha d'iniciar sessió correctament.
- El sistema ha de gestionar errors d'autenticació i codis incorrectes.

9.1.2. RF-02: Visualització i gestió de la pantalla Home

Història d'usuari

Com a usuari autenticat, vull visualitzar una pantalla Home amb informació rellevant del meu negoci, per poder consultar ràpidament l'estat de la meva activitat.

Criteris d'acceptació

- La pantalla Home ha de mostrar diversos widgets informatius independents.
- Cada widget ha de carregar les seves dades de forma autònoma.
- El sistema ha de mostrar estats de càrrega i error quan sigui necessari.
- La disposició visual ha de ser coherent amb el disseny establert.

9.1.3. RF-03: Assistència intel·ligent amb l'agent IA Cintia

Història d'usuari

Com a usuari de Localboss, vull rebre suggeriments intel·ligents de resposta a les reviews, per poder contestar-les de manera més ràpida, precisa i professional.

Criteris d'acceptació

- El sistema ha de detectar si l'usuari té activada la funcionalitat Cintia.
- Si Cintia està activa, el sistema ha de mostrar una suggerència de resposta.
- La suggerència s'ha de generar automàticament en obrir el modal de resposta.
- El sistema ha de mostrar la resposta de forma progressiva (efecte d'escriptura).
- L'usuari ha de poder editar o substituir la resposta suggerida.
- Si l'usuari no té credits Cintia se li comunicarà i tindrà un botó per compra-los.
- Si l'usuari no veu resposta tot i tenir crèdits, surtirà un botó per sol·licitar la resposta.

9.1.4. RF-04: Visualització de contingut multimèdia a les reviews

Història d'usuari

Com a usuari, vull poder veure imatges i vídeos associats a una review, per disposar de més context sobre l'opinió del client.

Criteris d'acceptació

- El sistema ha de mostrar les imatges i vídeos associats a una review.
- El contingut multimèdia s'ha de mostrar sota el text de la review.

- Les imatges han de ser navegables mitjançant scroll horitzontal.
- En clicar una imatge o vídeo, aquest s'ha de visualitzar a pantalla completa.

9.1.5. RF-05: Accés global al menú d'usuari

Història d'usuari

Com a usuari, vull poder accedir al menú de l'aplicació des de diferents pantalles, per facilitar la navegació i l'accés a les funcionalitats principals.

Criteris d'acceptació

- El menú ha d'estar accessible des de més d'una pantalla.
- L'accés al menú s'ha de fer mitjançant la imatge de perfil de l'usuari.
- El comportament del menú ha de ser coherent amb la resta de l'aplicació.

9.2. Model conceptual del domini

El model conceptual del domini defineix les principals entitats que intervenen en l'aplicació desenvolupada, així com les relacions existents entre elles. Aquest model permet entendre l'estructura lògica del sistema independentment de la seva implementació tècnica, centrant-se en els conceptes clau del negoci.

L'aplicació gira al voltant de la gestió de negocis locals i la interacció amb les ressenyes dels clients. A partir d'aquesta premissa, el domini s'ha modelat principalment mitjançant tres entitats centrals: **User**, **Location** i **Review**, a partir de les quals es deriven la resta de conceptes.

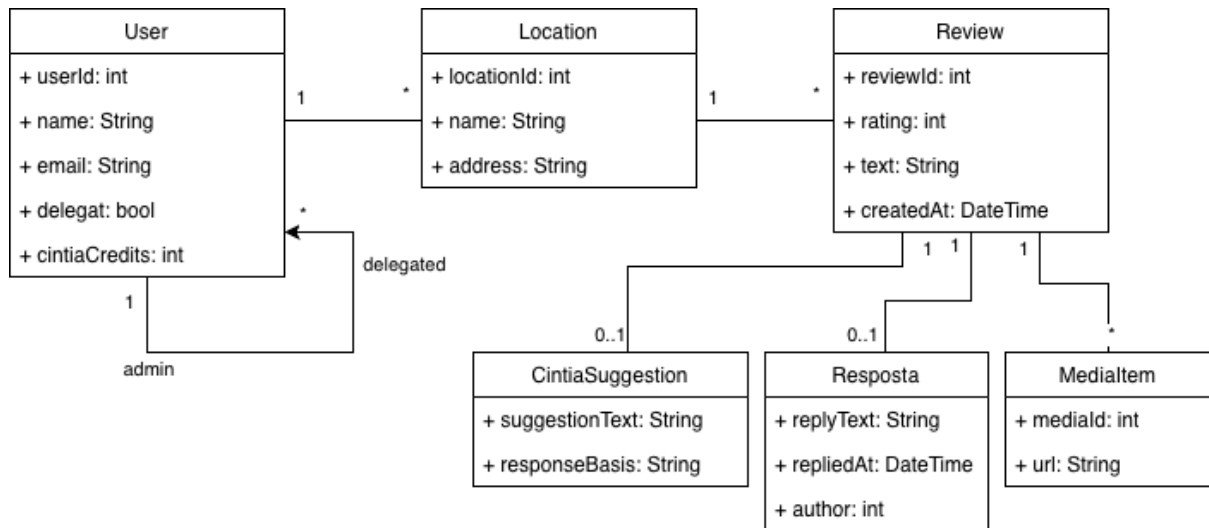


Figura 3: Model Conceptual

9.2.1. Entitat User

L'entitat **User** representa els usuaris de l'aplicació. Cada usuari disposa d'un identificador únic, dades bàsiques com el nom i el correu electrònic, i informació relacionada amb el seu rol dins del sistema. El camp `delegat` permet distingir entre usuaris administradors i usuaris delegats, mentre que `cintiaCredits` modela el nombre de crèdits disponibles per utilitzar la funcionalitat d'intel·ligència artificial.

A nivell conceptual, s'ha definit una **relació reflexiva** sobre aquesta mateixa entitat per representar la jerarquia entre usuaris. Un usuari administrador pot tenir associats zero o més usuaris delegats, mentre que cada usuari delegat està vinculat a un únic administrador. Aquesta relació permet modelar l'estructura de gestió d'equips dins l'aplicació.

9.2.2. Entitat Location

L'entitat **Location** representa els negocis o ubicacions gestionades dins de la plataforma. Cada ubicació disposa d'un identificador, un nom i una adreça. Un usuari administrador pot gestionar múltiples ubicacions, mentre que cada ubicació està associada a un únic usuari.

9.2.3. Entitat Review

L'entitat **Review** representa les ressenyes que els clients publiquen sobre una ubicació. Cada ressenya conté una valoració numèrica, un text descriptiu i la data de creació. Una ubicació pot tenir múltiples ressenyes, mentre que cada ressenya està associada a una única ubicació.

A partir de **Review** es deriven diversos elements relacionats amb la interacció i gestió de contingut:

- **CintiaSuggestion**: representa la suggerència generada per la intel·ligència artificial per respondre una ressenya. Aquesta entitat és opcional, ja que no totes les ressenyes disposen d'una proposta de resposta automàtica.
- **Resposta**: modela la resposta final publicada per l'usuari a una ressenya. També és opcional, ja que una ressenya pot no haver estat contestada.
- **MediaItem**: representa el contingut multimèdia associat a una ressenya, com imatges o vídeos. Una ressenya pot tenir múltiples elements multimèdia associats.

9.2.4. Visió global del domini

Aquest model conceptual permet representar de manera clara les relacions entre usuaris, negocis i ressenyes, així com la integració de funcionalitats avançades com la generació de respostes mitjançant intel·ligència artificial i la gestió de contingut multimèdia. El model serveix com a base per a la definició dels requisits i per a la posterior implementació del sistema.

9.3. Requisits no funcionals (VOLERE)

9.3.1. RNF-01: Usabilitat de l'aplicació

Camp	Descripció
Identificador del Requisit	RNF-01
Tipus de Requisit	Requisit no funcional – Usabilitat
Cas d'Ús / Event	Interacció general amb l'aplicació
Descripció	L'aplicació ha de proporcionar una interfície intuïtiva i coherent que permeti als usuaris completar les tasques principals (login, consulta i resposta de reviews, ús de Cintia) sense necessitat de formació prèvia.
Justificació del requisit	Localboss és una aplicació utilitzada per negocis reals; una mala usabilitat afectaria directament la productivitat i satisfacció dels usuaris.
Font	Empresa Localboss
Unitat en la que s'origina	Experiència d'usuari (UX)
Criteris de validació	Les funcionalitats principals poden ser utilitzades sense errors després d'un primer ús; no es requereixen instruccions externes per completar accions bàsiques.
Grau de satisfacció del interessat	Alt

Grau de insatisfacció del interessat	Alt
Dependencies	RNF-02 (Compatibilitat)
Conflictes	Cap
Documents de support	Dissenys UI/UX, feedback d'usuaris beta
Històric de canvis	Sense canvis
Projecte	Localboss
Analista	David

Taula 10: Volere de RNF-01

9.3.2. RNF-02: Compatibilitat amb dispositius mòbils

Camp	Descripció
Identificador del Requisit	RNF-02

Tipus de Requisit	Requisit no funcional – Compatibilitat
Cas d'Ús / Event	Execució de l'aplicació
Descripció	L'aplicació ha de funcionar correctament en dispositius Android i iOS amb diferents mides de pantalla i versions del sistema operatiu suportades.
Justificació del requisit	Els usuaris utilitzen una gran varietat de dispositius; garantir la compatibilitat evita errors visuals i funcionals.
Font	Empresa Localboss
Unitat en la que s'origina	Arquitectura Front-End
Criteris de validació	L'aplicació es visualitza i funciona correctament en emuladors i dispositius físics Android i iOS.
Grau de satisfacció del interessat	Alt
Grau de insatisfacció del interessat	Mitjà
Dependencies	RNF-01

Conflictes	Cap
Documents de support	Proves en emuladors i dispositius reals
Històric de canvis	Sense canvis
Projecte	Localboss
Analista	David

Taula 11: Volere de RNF-02

9.3.3. RNF-03 – Rendiment de l'aplicació

Camp	Descripció
Identificador del Requisit	RNF-03
Tipus de Requisit	Requisit no funcional – Rendiment
Cas d'Ús / Event	Consultes al backend

Descripció	Les respostes del sistema han de tenir un temps de resposta acceptable, evitant bloquejos o congelacions de la interfície d'usuari.
Justificació del requisit	Un rendiment deficient afecta negativament l'experiència d'usuari i la percepció de qualitat del producte.
Font	Empresa Localboss
Unitat en la que s'origina	Backend i comunicació REST
Criteris de validació	Les crides principals al backend retornen resposta en un temps percebut com acceptable per l'usuari.
Grau de satisfacció del interessat	Alt
Grau de insatisfacció del interessat	Alt
Dependencies	RNF-05 (Disponibilitat)
Conflictes	Cap
Documents de support	Logs d'execució, proves manuals

Històric de canvis	Sense canvis
Projecte	Localboss
Analista	David

Taula 12: Volere de RNF-03

9.3.4. RNF-04 – Seguretat de les dades

Camp	Descripció
Identificador del Requisit	RNF-04
Tipus de Requisit	Requisit no funcional – Seguretat
Cas d'Ús / Event	Autenticació i accés a dades
Descripció	Les dades dels usuaris han de ser transmeses i gestionades de forma segura, garantint que només els usuaris autoritzats puguin accedir a la informació.
Justificació del requisit	L'aplicació gestiona informació sensible de negocis i usuaris; la seguretat és crítica.

Font	Empresa Localboss
Unitat en la que s'origina	Backend
Criteris de validació	L'accés a funcionalitats requereix autenticació vàlida i no es permet accés no autoritzat.
Grau de satisfacció del interessat	Alt
Grau de insatisfacció del interessat	Alt
Dependencies	Requisits funcionals d'autenticació
Conflictes	Cap
Documents de support	Documentació de seguretat
Històric de canvis	Sense canvis
Projecte	Localboss
Analista	David

Taula 13: Volere de RNF-04

9.3.5. RNF-05 – Mantenibilitat del sistema

Camp	Descripció
Identificador del Requisit	RNF-05
Tipus de Requisit	Requisit no funcional – Mantenibilitat
Cas d'Ús / Event	Evolució del sistema
Descripció	El codi ha d'estar estructurat seguint bones pràctiques i patrons d'arquitectura que facilitin el manteniment i l'evolució del sistema.
Justificació del requisit	Localboss és un producte viu; cal facilitar futures ampliacions i correccions.
Font	Empresa Localboss
Unitat en la que s'origina	Arquitectura del sistema
Criteris de validació	El codi segueix una arquitectura clara i és revisable mitjançant PRs sense dificultat.

Grau de satisfacció del interessat	Alt
Grau de insatisfacció del interessat	Mitjà
Dependencies	Cap
Conflictes	Cap
Documents de support	Guia d'estil, revisió de codi
Històric de canvis	Sense canvis
Projecte	Localboss
Analista	David

Taula 14: Volere de RNF-05

10. Arquitectura i Disseny

Aquest capítol descriu l'arquitectura i el disseny del sistema desenvolupat durant el projecte, tant a nivell de front-end com de back-end. L'objectiu és explicar com s'ha estructurat tècnicament l'aplicació, quines decisions d'arquitectura s'han pres i quins patrons i tecnologies s'han utilitzat, justificats en el context d'un producte real en producció.

10.1. Visió general de l'arquitectura del sistema

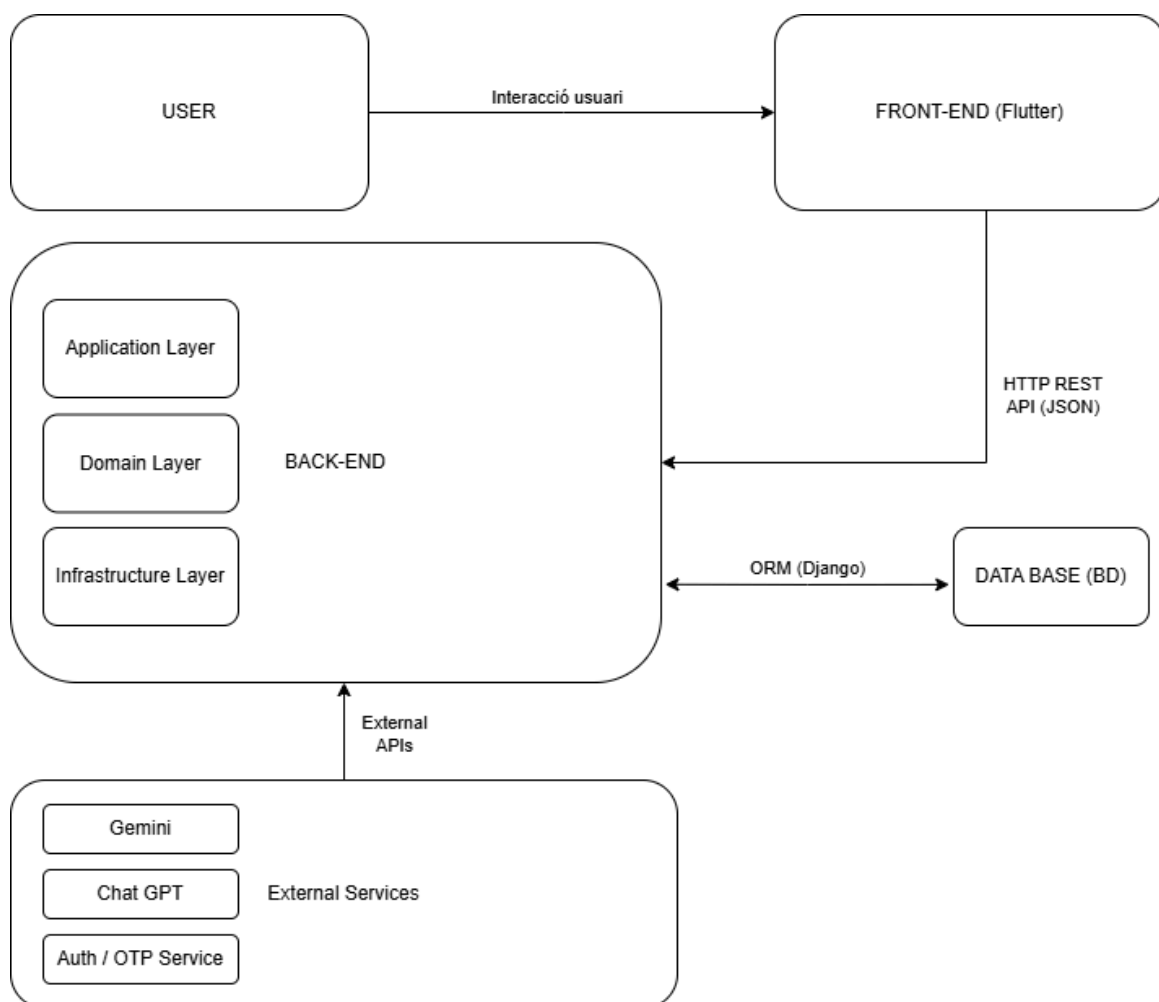


Figura 4: Arquitectura del sistema

Des del punt de vista de l'usuari, la interacció amb el sistema es realitza a través d'una aplicació mòbil desenvolupada amb Flutter, que actua com a capa de presentació (front-end). Aquesta aplicació s'encarrega de gestionar la interfície gràfica, la navegació

entre pantalles i la interacció amb l'usuari, així com de realitzar les peticions necessàries al back-end per obtenir o modificar informació. El front-end no conté lògica de negoci crítica, sinó que delega aquesta responsabilitat al servidor.

La comunicació entre el front-end i el back-end es realitza mitjançant una API REST, utilitzant missatges en format JSON tant per a les peticions com per a les respostes. Aquest mecanisme garanteix la independència tecnològica entre ambdues parts i permet que el sistema pugui ser consumit, si fos necessari, per altres clients en el futur.

El back-end està implementat utilitzant Django i segueix el patró d'Arquitectura Hexagonal (Ports and Adapters), que permet separar la lògica de negoci del framework i de les tecnologies d'infraestructura. Dins del back-end es poden distingir clarament tres capes principals:

- **Capa de Domini (Domain Layer):** conté la lògica de negoci més pura del sistema, així com els models de domini i les interfícies dels repositoris. Aquesta capa és totalment independent de Django, de la base de dades i de qualsevol servei extern.
- **Capa d'Aplicació (Application Layer):** inclou els casos d'ús del sistema, que orquestren la lògica de negoci i defineixen el flux de les operacions. Aquesta capa actua com a intermediària entre el domini i la infraestructura.
- **Capa d'Infraestructura (Infrastructure Layer):** és la responsable de la interacció amb l'exterior. En aquesta capa s'implementen les views HTTP de Django, els repositoris que accedeixen a la base de dades mitjançant l'ORM, i els adaptadors per a serveis externs.

La persistència de les dades es realitza mitjançant una base de dades relacional MySQL, gestionada a través de l'ORM de Django. Aquesta solució permet definir de manera clara les relacions entre entitats i garantir la integritat de les dades, així com facilitar l'evolució de l'esquema mitjançant migracions.

Finalment, el sistema integra diversos serveis externs a través del back-end, com ara serveis d'intel·ligència artificial (Gemini o ChatGPT) i serveis d'autenticació. Aquests serveis s'integren mitjançant adaptadors específics situats a la capa d'infraestructura, de manera que el domini del sistema no depèn directament de cap tecnologia externa.

En conjunt, aquesta arquitectura proporciona una base sòlida, flexible i escalable per al desenvolupament de l'aplicació, permetent incorporar noves funcionalitats amb un impacte mínim sobre el codi existent i assegurant una clara separació de responsabilitats entre les diferents parts del sistema.

10.2. Stack tecnològic

Les principals tecnologies utilitzades en el projecte són les següents:

- **Front-End:** Flutter (Dart), utilitzat per al desenvolupament multiplataforma de l'aplicació mòbil per a Android i iOS.
- **Back-End:** Django (Python), utilitzat com a framework principal per a la construcció de l'API REST.
- **Base de dades:** MySQL, com a sistema de gestió de bases de dades relacionals.
- **Integracions externes:**
 - Serveis d'intel·ligència artificial com Gemini i ChatGPT.
 - Firebase per a funcionalitats complementàries (notificacions, serveis associats).
- **Eines de desenvolupament:**
 - Jira per a la gestió de requisits i tasques.
 - Git com a sistema de control de versions.
 - Android Studio i VS Code com a entorns de desenvolupament.

Aquest stack ha estat escollit per coherència amb la infraestructura existent de Localboss i per garantir estabilitat, mantenibilitat i escalabilitat del producte.

10.3. Arquitectura del Front-End

10.3.1. Disseny extern: pantalles i navegació

El disseny extern de l'aplicació s'ha concebut amb l'objectiu d'oferir una experiència d'usuari clara, intuïtiva i coherent en dispositius mòbils, tenint en compte tant la jerarquia funcional de l'aplicació com els diferents contextos d'ús dels usuaris.

La navegació de l'app segueix principalment un **model jeràrquic**, en el qual l'usuari accedeix a les funcionalitats principals a partir d'un conjunt reduït de pantalles base. A partir d'aquestes pantalles, es despleguen fluxos específics segons l'acció realitzada, com ara la selecció d'una localització, la consulta de ressenyes o la resposta a una review concreta.

En funció del context, la navegació combina diferents mecanismes propis de Flutter:

- **Pantalles completes** (push/pop sobre el Navigator)
- **Modals i bottom sheets**, utilitzats per a accions contextuais o secundàries, com el delegated login o la resposta a reviews
- **Navegació condicional**, on el flux pot variar segons l'estat de l'usuari (per exemple, usuari administrador o delegat, disponibilitat de Cintia, etc.)

S'ha prestat especial atenció a la **coherència visual i funcional** entre pantalles, mantenint patrons de disseny consistents pel que fa a:

- Distribució dels elements
- Ús de colors i tipografia
- Comportament dels botons i components interactius

A més, totes les pantalles s'han dissenyat pensant en la **responsivitat**, garantint una correcta adaptació a diferents mides i resolucions de pantalla, tant en dispositius Android com iOS.

A continuació, un petit resum visual de les diverses pantalles i la seva navegació

10.3.1.1. Disseny visual final

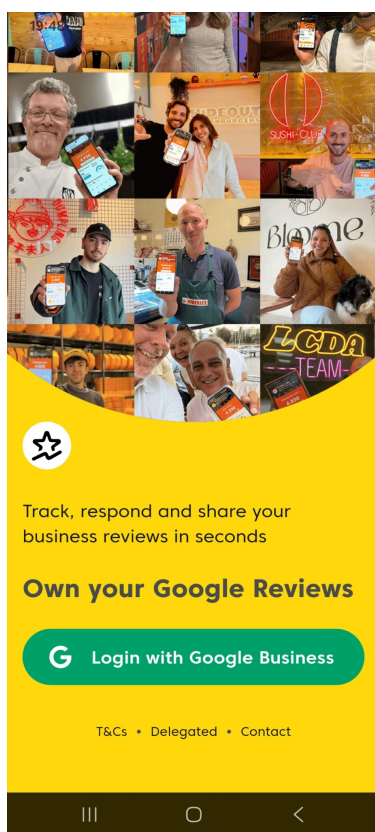


Figura 5: Pantalla login

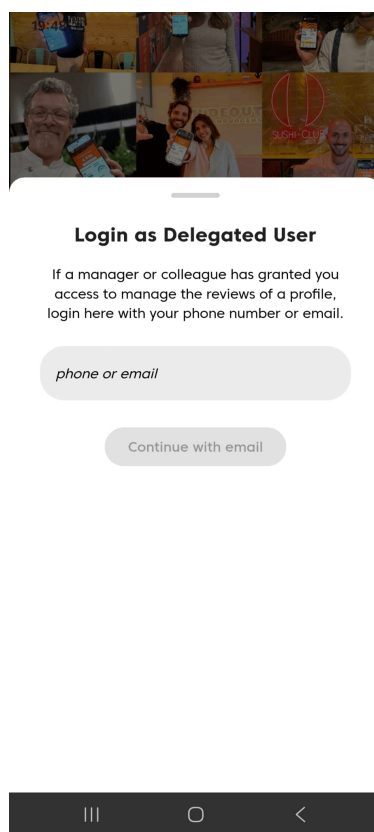


Figura 6: Pantalla delegated login 1

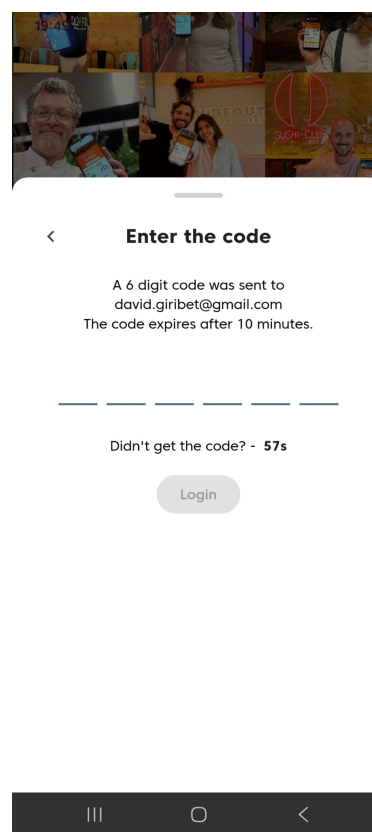


Figura 7: Pantalla delegated login 2

- **Benvinguda / Login (Figura 5):**
 - **Descripció:** Aquesta interfície constitueix el punt d'entrada inicial a l'aplicació. Presenta les opcions d'autenticació disponibles, destacant el mètode principal mitjançant Google OAuth 2.0. A la part inferior, s'inclou un accés secundari per a la modalitat d'usuari delegat.
 - **Implementació pròpia:** No directament en el botó de Google, però s'ha modificat la interfície per incloure i canalitzar el flux cap al **Delegated Login**, assegurant la coexistència dels dos sistemes d'accés.
- **Pantalla d'accés per a Delegats (Figura 6):**
 - **Descripció:** Interfície dissenyada per als usuaris que no disposen de credencials directes de Google Business Profile. Sol·licita l'identificador de la localització o el correu electrònic per iniciar el procés de verificació d'identitat alternatiu.
 - **Implementació pròpia:** Aquesta pantalla ha estat **desenvolupada íntegrament** com a part de la nova arquitectura d'accessos. S'ha programat la validació dels camps d'entrada i la comunicació amb l'API per verificar la validesa de la instància abans de procedir al següent pas.
- **Pantalla de verificació OTP (Figura 7):**
 - **Descripció:** Pantalla de seguretat on l'usuari ha d'introduir el codi temporal (One-Time Password) rebut per correu electrònic. Presenta un

camp segmentat per a sis dígits, optimitzat per a la introducció ràpida de dades.

- **Implementació pròpia:** S'ha implementat tant el disseny de la interfície (amb gestió automàtica del focus entre dígits) com la **lògica de control de l'estat (BLoC)**. Aquesta gestiona els intents d'accés, la validació del codi contra el back-end i la redirecció final de l'usuari un cop autenticat correctament.

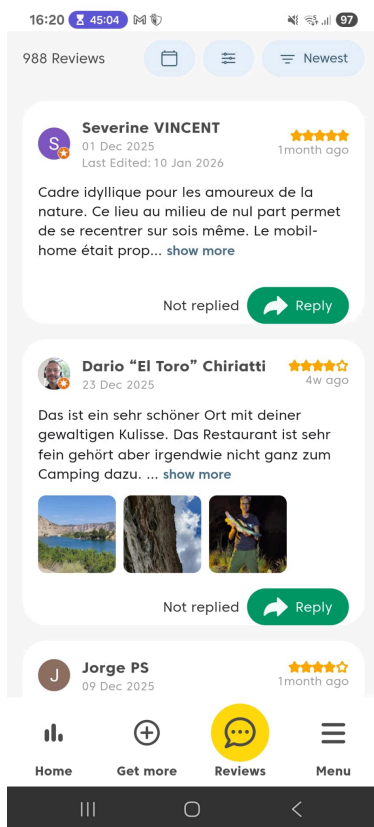


Figura 8: Pantalla de Reviews

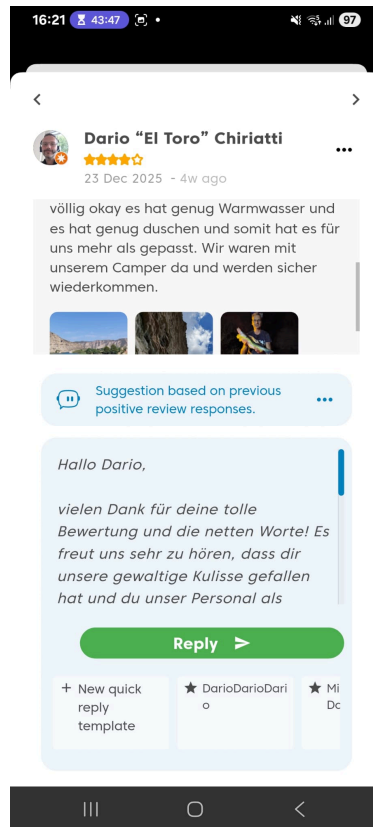


Figura 9: Pantalla del modal de resposta 1

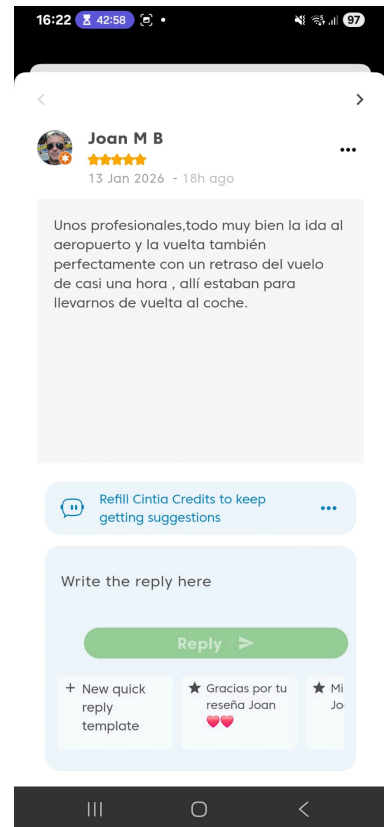


Figura 10: Pantalla del modal de resposta 2

- **Llistat de Ressenyes (Reviews) (Figura 8):**
 - **Descripció:** Interfície que recull l'històric de valoracions dels clients. Cada entrada mostra la puntuació (estrelles), el nom de l'usuari, la data i el cos del text de la ressenya. Està dissenyada per oferir una visió ràpida del feedback rebut pel negoci.
 - **Implementació pròpia:** S'ha integrat la funcionalitat de **contingut multimèdia**. L'aplicació ara és capaç de detectar, carregar i mostrar les imatges adjuntes a les ressenyes (com s'observa en els thumbnails de la imatge), així com la visualització de vídeos.
- **Modal de Resposta (Agent Cintia) (Figura 9):**

- **Descripció:** Pantalla emergent (*bottom sheet*) que s'activa quan l'usuari decideix respondre una ressenya específica. Inclou un camp de text per a la redacció manual i l'accés a l'assistent d'intel·ligència artificial.
- **Implementació pròpia:** S'ha implementat tota la lògica d'interacció amb l'**Agent Cintia**. Això inclou el botó d'invocació de la IA, la recepció del suggeriment de resposta basat en el text de la ressenya i la càrrega automàtica d'aquest contingut al camp de text per a la seva revisió final. A més a més, de igual forma que amb la figura anterior, també s'ha implementat el contingut multimedia a sota del text.
- **Gestió de Crèdits de Cintia (Figura 10):**
 - **Descripció:** Variant del modal de resposta que es mostra quan un usuari ha esgotat el seu límit de consultes d'IA. Presenta un disseny informatiu i restrictiu que impedeix l'ús de la funcionalitat fins a l'adquisició de nous crèdits.
 - **Implementació pròpia:** S'ha desenvolupat la **lògica de control de crèdits** mitjançant la gestió d'estats. La interfície reacciona dinàmicament a l'estat de la subscripció de l'usuari, bloquejant el camp de generació i mostrant un missatge contextual d'error.

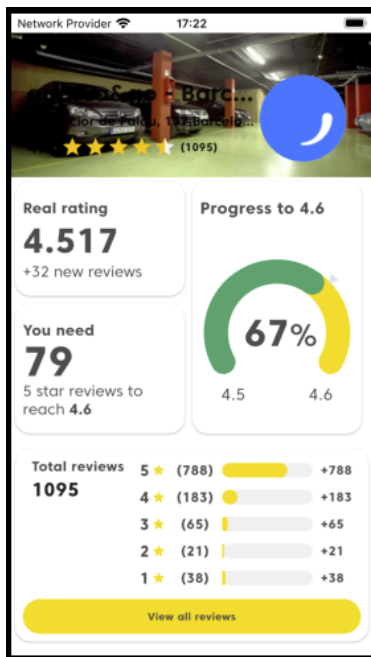


Figura 11: Pantalla nova home 1

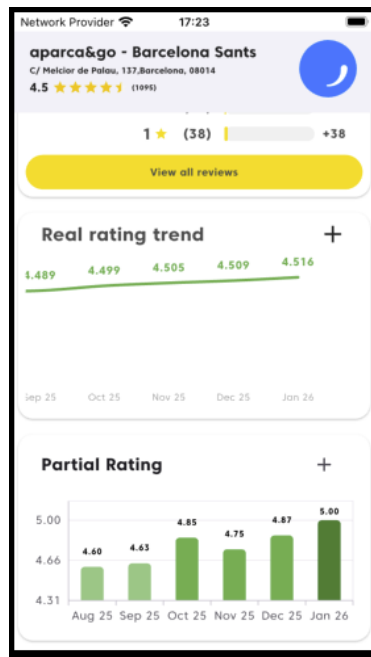


Figura 12: Pantalla nova home 2

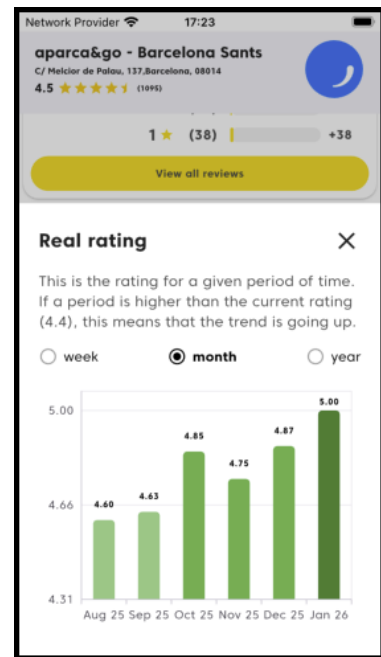


Figura 13: Pantalla nova home 3

- **Home Vista Estesa (Figura 11):**
 - **Descripció:** Aquesta interfície representa el panell de control principal (home) de l'aplicació. En la seva vista estesa, presenta una SliverAppBar que mostra la informació del negoci seleccionat, la nota mitjana actual i

el "Real Rating". A sota, s'organitzen diversos mòduls d'estadístiques i gràfics que resumeixen l'estat de la reputació online del negoci.

- **Implementació pròpia:** Implementació completa de la pantalla.
- **Home Vista Comprimida (Scroll) (Figura 12):**
 - **Descripció:** Variant de la pantalla principal que s'activa quan l'usuari realitza un desplaçament vertical (scroll down). La AppBar es redueix per maximitzar l'espai útil de lectura dels gràfics d'evolució, mantenint visibles els selectors temporals i les mètriques de rendiment.
 - **Implementació pròpia:** Implementació completa de la pantalla.
- **Home Detall de Mètriques (Expandit) (Figura 13):**
 - **Descripció:** Interfície que es desplega en interactuar amb el botó d'expansió ("+") dels gràfics. Aquesta vista ofereix una anàlisi detallada d'una mètrica específica (com el rànking o l'evolució de ressenyes), permetent a l'usuari aprofundir en les dades històriques sense abandonar el context de la pantalla principal.
 - **Implementació pròpia:** Implementació completa de la pantalla.

10.3.1.2. Flux de navegació

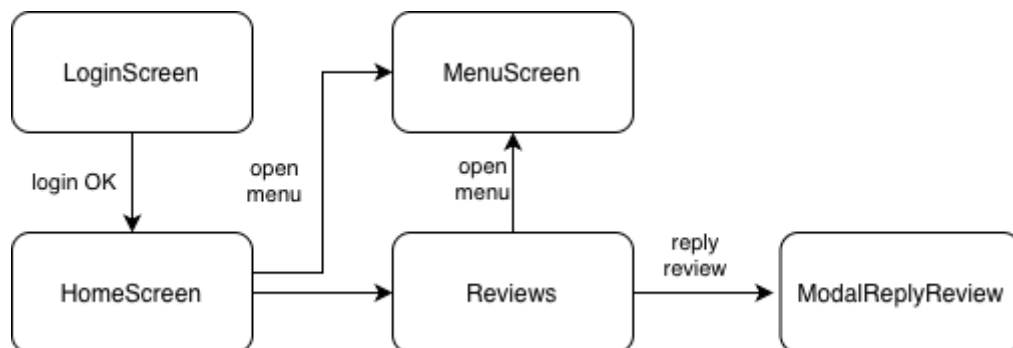


Figura 14: Diagrama de classes indicant navegació

El diagrama de flux de navegació presenta una estructura organitzada per nivells de profunditat i estats d'autenticació, facilitant una experiència d'usuari fluida i sense punts cecs. Es poden identificar tres blocs principals:

- **LoginScreen:** El flux s'inicia amb la pantalla de **Login**, que actua com a node de decisió. Un cop validat l'usuari, s'accedeix a la **Home**, on pot veure la informació bàsica del seu negoci.
- **HomeScreen:** La pantalla **Home** (totalment refactoritzada en aquest projecte) centralitza l'activitat. Des d'aquí, la navegació és radial:
 - **Navegació cap les Reviews:** Accés a la llista completa de **Reviews**.

- **Accés al menú:** Accés a la pantalla on l'usuari pot veure la seva informació personal, informació de la app, i pot fer logout.
- **Reviews:** Pantalla més important de l'aplicació, on l'usuari pot veure totes les seves reviews, ordenar-les o filtrar-les segons preferència, o obrir una per respondre-la.
- **ModalReplyReviews:** Dins del flux de ressenyes, s'arriba al detall de la **Review**, que és el punt de més interacció. Aquí és on s'integren els mòduls crítics treballats:
 - **Cintia (IA):** El flux permet invocar l'agent d'IA per generar respostes.
 - **Multimèdia:** Visualització d'imatges i vídeos vinculats a la ressenya.
 - **Flux de resposta:** Un cop generada o escrita la resposta, el flux es tanca retornant l'usuari a la llista actualitzada.

10.3.2. Disseny intern: components i patrons de disseny

A nivell intern, el front-end de l'aplicació segueix una arquitectura modular basada en **features**, on el codi s'organitza per funcionalitats en lloc de fer-ho per tipus de components. Aquest enfocament permet encapsular tot el que està relacionat amb una funcionalitat concreta (pantalles, widgets, cubits, estats i serveis associats) dins d'un mateix mòdul, facilitant així el manteniment i l'escalabilitat del projecte.

Per a la gestió de l'estat i de la lògica de negoci s'utilitza el patró **Bloc**, concretament mitjançant la seva variant **Cubit**, àmpliament utilitzada en aplicacions Flutter modernes. Aquest patró permet una separació clara entre la capa de presentació i la lògica d'aplicació, evitant que les pantalles continguin codi de negoci o dependències directes amb el backend.

10.3.2.1. Gestió de l'estat amb Bloc/Cubit

Cada funcionalitat rellevant disposa del seu propi Cubit, responsable de:

- Executar les accions necessàries (crides a casos d'ús o serveis)
- Gestionar les transicions d'estat
- Exposar l'estat actual a la interfície d'usuari

El funcionament general d'un Cubit dins de l'aplicació segueix el següent esquema:

- La pantalla o widget inicialitza el Cubit corresponent.
- Quan l'usuari realitza una acció (per exemple, carregar dades, enviar un formulari o demanar una resposta de Cintia), la pantalla invoca un mètode del Cubit.
- El Cubit executa la lògica necessària, que pot incloure crides a serveis, repositoris o casos d'ús.
- En funció del resultat obtingut, el Cubit emet un nou estat.
- La interfície d'usuari reacciona automàticament al canvi d'estat, mostrant el contingut adequat.

Aquesta comunicació es basa en un flux **unidireccional**, fet que millora la previsibilitat del comportament de l'aplicació i redueix els efectes col·laterals.

Gestió d'estats: Els Cubits defineixen diferents estats per representar les fases d'una operació. Els estats que pots definir són els que vulguis i poden variar en funció del que es necessiti, però hi ha alguns més comuns, com ara:

- **Estat inicial**, on s'inicialitza el cubit i no es mostra informació
- **Estat de càrrega (loading)**, utilitzat mentre s'espera una resposta del backend
- **Estat d'èxit (success)**, que conté les dades necessàries per renderitzar la interfície
- **Estat d'error**, que permet mostrar missatges d'error o feedback a l'usuari

Aquest enfocament s'ha aplicat de manera sistemàtica a tota l'aplicació, especialment en funcionalitats complexes com:

- El delegated login, que requereix la coordinació de múltiples crides al backend
- La nova pantalla Home, amb diversos widgets independents i fonts de dades diferents

- La integració de l'agent d'intel·ligència artificial Cintia, on el contingut es genera de forma asíncrona i progressiva

Beneficis de l'enfocament adoptat: L'ús combinat d'una arquitectura basada en features i del patró Bloc/Cubit aporta diversos avantatges:

- Millora del manteniment del codi
- Facilitat per afegir noves funcionalitats sense afectar les existents
- Separació clara de responsabilitats
- Millor testabilitat de la lògica d'aplicació
- Reducció de dependències entre components

Aquest disseny intern ha estat clau per poder desenvolupar funcionalitats de gran complexitat dins d'un entorn professional real, garantint alhora la qualitat, l'estabilitat i l'escalabilitat de l'aplicació.

10.4. Arquitectura del Back-End

El back-end del projecte segueix el patró d'**Arquitectura Hexagonal**, que permet desacoblar completament la lògica de negoci del framework utilitzat.

10.4.1. Arquitectura hexagonal

L'Arquitectura Hexagonal permet definir un nucli de domini independent de la infraestructura. Les dependències sempre apunten cap al domini, evitant que aquest depengui de detalls externs com l'ORM, el framework web o serveis de tercers.

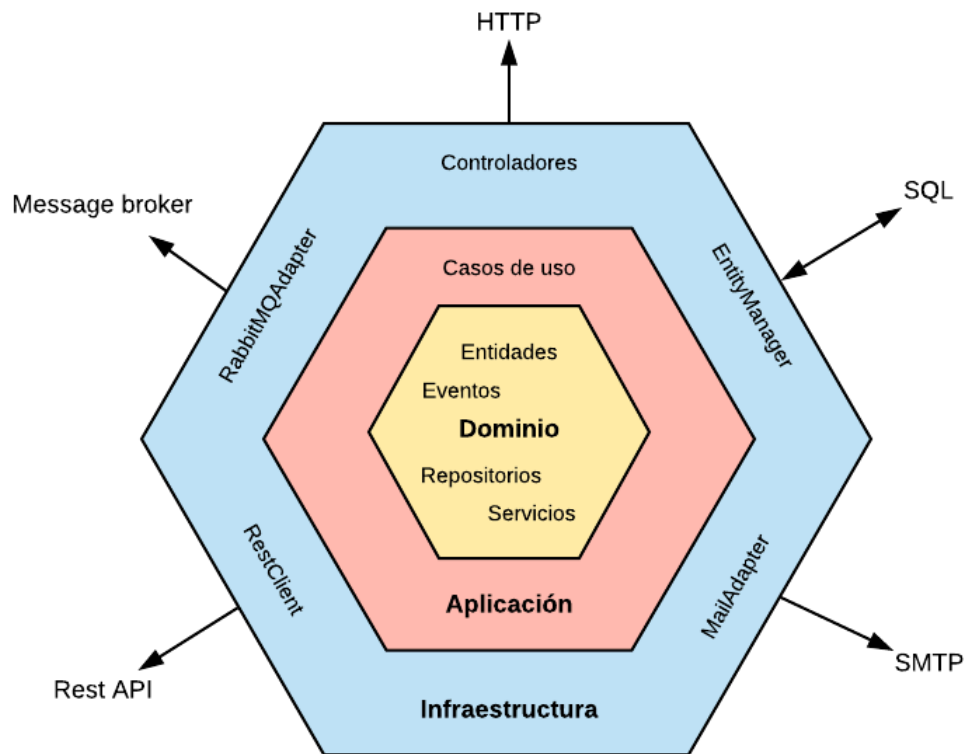


Figura 15: Arquitectura Hexagonal [\[R14\]](#)

10.4.2. Descripció de les capes

- **Capa de Domini (domain):** Conté la lògica de negoci més pura del sistema. Defineix els models de domini i les interfícies dels repositoris, sense cap dependència de Django ni de la base de dades.
- **Capa d'Aplicació (application):** Conté els casos d'ús del sistema. Aquesta capa coordina la lògica de negoci, orquestra les operacions i utilitza els ports definits al domini per accedir a les dades.
- **Capa d'Infraestructura (infrastructure):** Implementa els detalls tècnics del sistema. Inclou:
 - Les views HTTP de Django, que actuen com a controladors.
 - Les implementacions dels repositoris mitjançant l'ORM de Django.
 - Les integracions amb serveis externs com Gemini, ChatGPT o Firebase.

10.4.3. Patrons de disseny utilitzats

- **Inversió de Dependència (DIP):** El domini defineix interfícies que són implementades per la infraestructura.
- **Repository Pattern:** Aïlla l'accés a dades i simplifica l'ús de l'ORM als casos d'ús.
- **Factory Pattern:** Utilitzat per instanciar dinàmicament serveis d'intel·ligència artificial segons la configuració del sistema.

10.4.4. Disseny de la base de dades

La base de dades utilitza **MySQL** com a motor relacional. Les taules i relacions es defineixen mitjançant l'ORM de Django, permetent una gestió senzilla de migracions i assegurant la coherència de l'esquema.

S'utilitzen relacions d'integritat referencial, principalment de tipus 1:N, com ara entre usuaris, negocis i reviews.

10.4.5. Comunicació Front-End / Back-End

La comunicació es realitza mitjançant una API REST basada en HTTP. Les dades s'intercanvien en format JSON, utilitzant mètodes HTTP estàndard com GET i POST.

El front-end és totalment independent del back-end, coneixent únicament l'estructura dels endpoints i el format de les dades.

10.5. Diagrames de seqüència

Els diagrames de seqüència permeten representar de manera detallada la interacció temporal entre els diferents components del sistema durant l'execució d'una funcionalitat concreta. A través d'aquests diagrames es mostra l'ordre en què es produeixen les

comunicacions entre actors, front-end, back-end i serveis externs, facilitant la comprensió del flux complet d'una operació des del punt de vista arquitectònic.

En aquest projecte s'han seleccionat dues funcionalitats representatives per a la seva modelització mitjançant diagrames de seqüència. La primera correspon al procés d'autenticació mitjançant **Delegated Login amb codi OTP**, ja que implica múltiples interaccions entre diferents serveis. La segona correspon a la funcionalitat de l'agent d'intel·ligència artificial **Cintia**, una de les aportacions principals del projecte, que integra serveis externs d'IA dins del flux de gestió de reviews.

10.5.1. Diagrama de seqüència del Delegated Login (autenticació amb OTP)

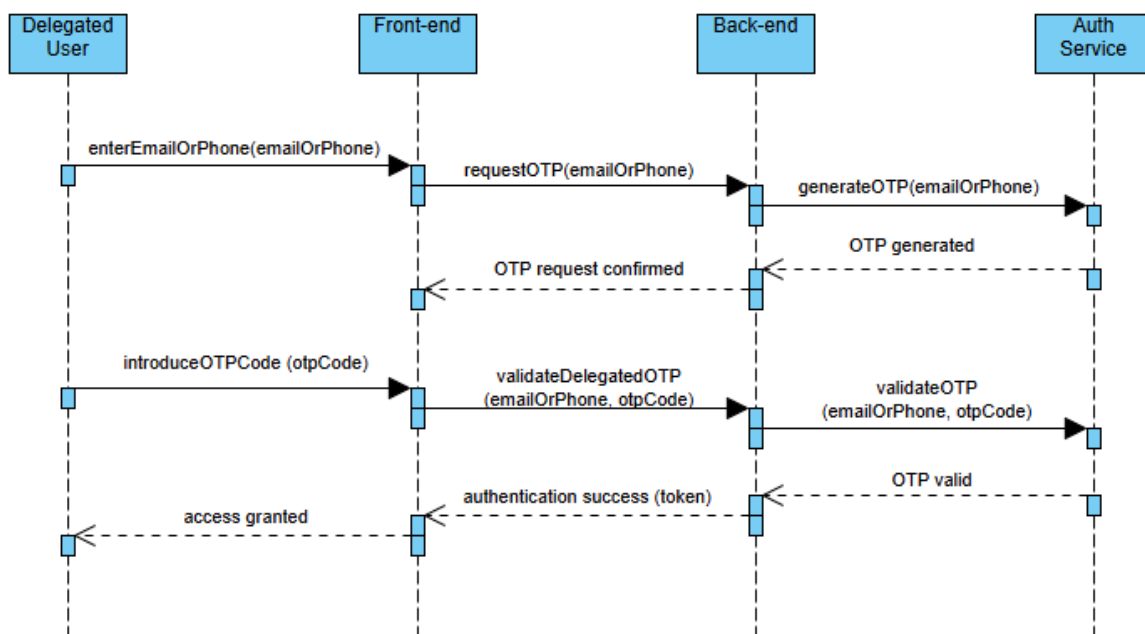


Figura 16: Diagrama Seqüència Delegated Login

Aquest diagrama descriu el procés d'autenticació d'un usuari delegat mitjançant un codi d'un sol ús (OTP). Aquesta funcionalitat permet a usuaris no administradors accedir a l'aplicació sense necessitat de contrasenya, reforçant la seguretat i simplificant el procés de login.

El flux s'inicia quan l'usuari delegat introdueix el seu correu electrònic o número de telèfon a la interfície de l'aplicació. El front-end envia aquesta informació al back-end mitjançant una petició de sol·licitud d'OTP. El back-end, al seu torn, delega la generació del codi a un servei d'autenticació especialitzat, encarregat de crear i enviar l'OTP a l'usuari.

Un cop el codi ha estat generat i enviat, el servei d'autenticació confirma al back-end que l'OTP s'ha creat correctament. Aquesta confirmació permet al front-end informar l'usuari que pot introduir el codi rebut. Quan l'usuari introdueix l'OTP, el front-end envia una nova petició al back-end per validar-lo.

El back-end torna a comunicar-se amb el servei d'autenticació per comprovar la validesa del codi introduït. Si l'OTP és correcte, el servei retorna una confirmació positiva, i el back-end finalitza el procés d'autenticació concedint accés a l'usuari. Finalment, el front-end notifica a l'usuari que l'autenticació ha estat satisfactòria i permet l'accés a l'aplicació.

Aquest diagrama mostra clarament el desacoblament entre el front-end i el servei d'autenticació, així com el paper del back-end com a element central de coordinació del procés.

10.5.2. Diagrama de seqüència de la generació de suggeriments amb Cintia

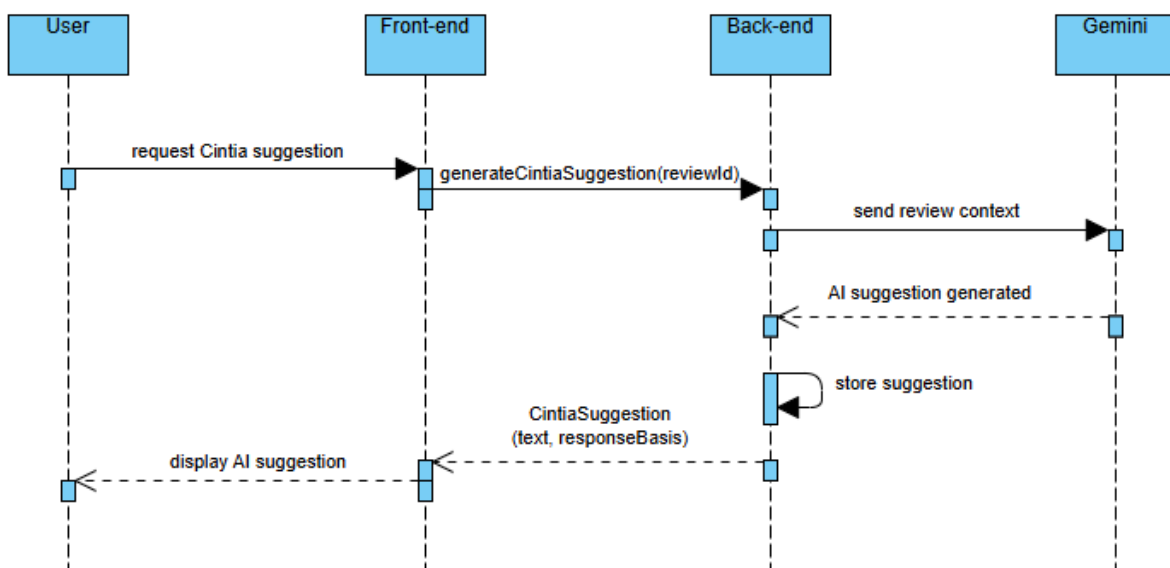


Figura 17: Diagrama Seqüència Cintia

Aquest diagrama representa el funcionament de la funcionalitat d'intel·ligència artificial **Cintia**, encarregada de generar suggeriments de resposta per a les reviews dels negocis. Aquesta funcionalitat integra un servei extern d'IA dins del sistema, mantenint la coherència amb l'arquitectura definida.

El procés s'inicia quan l'usuari sol·licita un suggeriment de resposta per a una review concreta des de la interfície de l'aplicació. El front-end envia aquesta sol·licitud al back-end, indicant l'identificador de la review corresponent.

El back-end recupera el context necessari de la review (text, valoració i informació rellevant) i envia aquesta informació al servei d'intel·ligència artificial, en aquest cas Gemini. El servei d'IA processa el contingut rebut i genera una proposta de resposta basada en el context proporcionat.

Un cop el suggeriment ha estat generat, el servei d'IA retorna el resultat al back-end. Aquest, abans de respondre al front-end, emmagatzema el suggeriment dins del sistema, associant-lo a la review corresponent. Aquest pas permet mantenir la persistència de la informació i reutilitzar el suggeriment en futures interaccions.

Finalment, el back-end envia el suggeriment al front-end, que el mostra a l'usuari dins del modal de resposta. Aquest flux garanteix que la generació de contingut mitjançant IA s'integra de manera transparent dins del sistema, sense que el front-end hagi d'interactuar directament amb serveis externs.

Aquest diagrama posa de manifest la integració d'un servei d'intel·ligència artificial dins d'una arquitectura desacoplada, així com el paper del back-end en la gestió de la lògica de negoci i la persistència de dades.

11. Implementació

Aquest capítol descriu com s'ha dut a terme la implementació del projecte des d'un punt de vista tècnic. L'objectiu no és detallar funcionalitats concretes, ja descrites en capítols anteriors, sinó explicar com s'ha organitzat el codi, quines arquitectures s'han aplicat a nivell pràctic, quines llibreries i eines s'han utilitzat i quines decisions tècniques han estat rellevants durant el desenvolupament.

El projecte s'ha desenvolupat en un entorn professional real, fet que ha condicionat moltes decisions d'implementació, prioritzant la qualitat del codi, la mantenibilitat, l'escalabilitat i la integració amb un sistema ja existent en producció.

11.1. Organització general del projecte

El projecte està dividit en dos grans blocs clarament diferenciats:

- **Front-end**, desenvolupat amb Flutter, responsable de la interfície d'usuari i de la interacció amb l'usuari final.
- **Back-end**, desenvolupat amb Django, encarregat de la lògica de negoci, la persistència de dades i la integració amb serveis externs.

Ambdós blocs es troben en **repositoris independents**, fet que permet una evolució desacoblada de cada part del sistema. La comunicació entre el front-end i el back-end es realitza mitjançant una **API REST**, utilitzant el protocol HTTP i intercanviant dades en format JSON.

Aquesta separació facilita el manteniment del sistema, permet treballar de manera paral·lela i redueix l'impacte dels canvis en una de les parts sobre l'altra.

Aquest treball tracta sobre el desenvolupament a Front-end, per tant, es farà èmfasi en el desenvolupament d'aquest, i no tant en la implementació del back.

11.2. Organització del codi

11.2.1. Front-end

El front-end està implementat amb Flutter i segueix una estructura basada en els principis de la **Clean Architecture**, aplicada de manera estricta. Aquesta arquitectura es reflecteix directament en l'organització del codi, separant clarament les responsabilitats en diferents capes:

- **Presentation:** gestiona l'estat de l'aplicació i la comunicació amb la interfície d'usuari mitjançant Bloc/Cubit.
- **Domain:** conté la lògica de negoci, amb use cases, entitats i repositoris definits com a interfícies.
- **Data:** s'encarrega de l'accés a dades, incloent les crides al backend, la transformació de dades i la implementació concreta dels repositoris.

Les pantalles de l'aplicació es troben principalment a la carpeta screens, mentre que funcionalitats de gran envergadura, com el refactor de la Home, s'organitzen dins de la carpeta features, agrupant en un mateix lloc tots els elements relacionats amb una funcionalitat concreta.

Aquesta estructura permet un alt grau de modularitat, facilita la lectura del codi i millora la seva escalabilitat.

11.2.2. Back-end

El back-end està desenvolupat amb Django i segueix el patró d'**Arquitectura Hexagonal**, que permet desacoplar la lògica de negoci del framework i de les tecnologies d'infraestructura.

L'organització del codi es divideix principalment en:

- **Capa de domini (domain):** conté els models de negoci i els repositoris definits com a interfícies, sense dependències de Django ni de la base de

dades.

- **Capa d'aplicació (application):** inclou els casos d'ús, que coordinen la lògica de negoci i fan ús dels repositoris del domini.
- **Capa d'infraestructura (infrastructure):** implementa els repositoris utilitzant l'ORM de Django, defineix les views HTTP i integra serveis externs com sistemes d'intel·ligència artificial.

Aquesta estructura garanteix que la lògica de negoci sigui independent del framework i facilita la seva evolució i testabilitat.

11.3. Arquitectura aplicada a nivell d'implementació

Les decisions arquitectòniques definides en el capítol d'Arquitectura i Disseny tenen una aplicació directa en el codi del projecte.

En el front-end, la comunicació entre la interfície d'usuari i la lògica de negoci es realitza mitjançant Cubits, que invoquen use cases del domini. Aquests use cases accedeixen als repositoris, que obtenen les dades des del backend a través de la capa de dades.

En el back-end, les peticions HTTP rebudes per les views de Django es deleguen als casos d'ús corresponents, que encapsulen la lògica de negoci i utilitzen repositoris per accedir a la base de dades o, a serveis externs.

Aquest flux assegura un **desacoblament complet entre capes**, evitant dependències directes entre la interfície i els detalls d'implementació.

11.4. Llibreries i serveis externs

11.4.1. Llibreries Flutter

Durant el desenvolupament del projecte s'han incorporat diverses llibreries externes per implementar funcionalitats específiques:

- **syncfusion_flutter_charts**: utilitzada per a la creació de gràfics avançats a la nova pantalla Home, substituint paquets anteriors desactualitzats.
- **fl_chart**: emprada per a la implementació de gràfics lineals, funcionalitat no suportada pels paquets anteriors.
- **SliverAppBar**: utilitzada per implementar una App Bar avançada amb comportament dinàmic en funció de l'scroll.
- **video_player**: incorporada per permetre la reproducció de vídeos associats a les reviews.

Aquestes llibreries han estat seleccionades després d'un procés d'anàlisi per garantir compatibilitat, manteniment actiu i adequació a les necessitats del projecte.

11.4.2. Serveis externs

El projecte integra diversos serveis externs:

- **Serveis d'intel·ligència artificial**, com Gemini i ChatGPT, utilitzats per a la funcionalitat Cintia.
- **Serveis d'autenticació**, com el login amb Google.
- **Firebase**, utilitzat en parts concretes del sistema heretades de versions anteriors.

La integració d'aquests serveis es realitza exclusivament des del backend, mantenint el front-end totalment desacoblat dels detalls d'implementació.

11.5. Eines de desenvolupament i qualitat del codi

El desenvolupament s'ha realitzat utilitzant eines professionals:

- **Android Studio i VS Code** com a entorns de desenvolupament.
- **Postman** per provar i validar els endpoints del backend abans de la seva integració al front-end.
- **GitHub** per a la gestió del codi, pull requests i revisions.
- **Linters i checks automàtics** configurats al repositori per garantir el compliment d'estàndards de qualitat abans de permetre el merge a la branca principal.

Aquest conjunt d'eines ha contribuït a mantenir un alt nivell de qualitat i consistència en el codi.

11.6. Gestió de repositoris i branques

El projecte utilitza una estratègia de branques basada en:

- Una branca principal (main) estable.
- Una branca per a cada funcionalitat o tasca (feature/...).
- Creació de **Pull Requests** un cop finalitzat el desenvolupament.
- Revisions de codi abans de fer el merge.

Aquesta estratègia està integrada amb Jira, permetent traçabilitat entre requisits, tasques i codi.

11.7. Decisions tècniques rellevants

Algunes de les decisions tècniques més rellevants han estat:

- L'ús de **Bloc/Cubit** per a la gestió de l'estat en lloc de solucions més simples.
- L'aplicació estricta de **Clean Architecture** al front-end.
- L'ús d'**Arquitectura Hexagonal** al back-end.
- La reutilització i adaptació d'endpoints existents en lloc de crear-ne de nous quan ha estat possible.
- La substitució de llibreries obsoletes per alternatives modernes i mantingudes.

Aquestes decisions han estat clau per garantir la qualitat i sostenibilitat del projecte.

11.8. Exemple d'implementació rellevant: Delegated Login

En aquest apartat es detalla la implementació del mòdul de Delegated Login, una funcionalitat crítica dissenyada per estendre l'accessibilitat de l'aplicació a usuaris sense comptes de proveïdors externs (Google). S'ha escollit aquest component com a exemple d'estudi per la seva complexitat i per representar de manera integral el flux de dades en l'arquitectura del projecte: des de la captura i validació d'inputs en la capa de presentació, fins a la gestió d'estats asíncrons mitjançant el patró BLoC/Cubit i la comunicació amb l'API de backend. El codi que es presenta a continuació reflecteix el compromís amb la robustesa, la seguretat en l'enviament de credencials efímeres (OTP) i una experiència d'usuari fluida i altament reactiva.

11.8.1. Implementació de la Capa de Dades: Remote Data Source

```
@override
Future<bool> loginDelegatedRequestByPhone(String phone) async {
  try {
    final data = {'phone_number': phone};

    Response? response = await _authServiceBaseApi.post(
      "${ApiMappings.loginDelegated}/login",
      data: data,
    );

    if (response != null && response.statusCode == 200) {
      return true;
    }
    return false;
  } catch (e) {
    throw ServerException();
  }
}
```

Figura 18: Codi Request OTP code

En aquesta primera captura, s'observa el mètode `loginDelegatedRequestByPhone`. Aquesta funció rep el número de telèfon com a paràmetre i realitza una petició de tipus POST cap a la ruta configurada en `ApiMappings`.

- **Preparació de la trama:** Es construeix un objecte JSON amb la clau `phone_number`.
- **Execució i control:** Es realitza la crida mitjançant el client API i, en cas de rebre un status code 200, el mètode retorna un booleà exitós (`true`), validant que el backend ha processat l'enviament de l'OTP. Qualsevol error de xarxa o de servidor és capturat i reemès com una `ServerException`.

```

@Override
Future<LoginResponseModel?> loginDelegatedValidateByPhone(
    String phone,
    String otpCode,
) async {
    try {
        final data = {'phone_number': phone, 'otp_code': otpCode};

        Response? response = await _authServiceBaseApi.post(
            "${ApiMappings.loginDelegated}/validate-otp",
            data: data,
        );

        if (response != null && response.statusCode == 200) {
            final loginModelResponse = LoginResponseModel.fromJson(response.data);
            await _storage.saveSecureData(
                'lbApiToken',
                loginModelResponse.accessToken,
            );
            await _storage.saveSecureData(
                'lbRefreshToken',
                loginModelResponse.refreshToken,
            );
            return loginModelResponse;
        }
        return null;
    } catch (e) {
        throw ServerException();
    }
}

```

Figura 19: Codi Validate OTP code

Aquesta segona captura mostra el mètode `loginDelegatedValidateByPhone`, el qual gestiona la segona part del flux d'autenticació.

- **Consum i mapeig:** S'envia un objecte JSON amb el telèfon i el codi OTP rebut per l'usuari. Si la resposta és satisfactòria, les dades en cru del backend es transformen automàticament en un objecte de tipus `LoginResponseModel`.
- **Gestió de la sessió:** Abans de retornar el model a les capes superiors, el mètode gestiona de forma interna la persistència de seguretat. S'emmagatzemen tant l'`accessToken` com el `refreshToken` a l'emmagatzematge segur del dispositiu sota les claus `lbApiToken` i `lbRefreshToken`, garantint la traçabilitat de la sessió.

11.8.2. Gestió de la lògica de negoci (Cubit)

```
Future<void> sendLoginDelegatedRequestByPhone(String id) async {
    emit(LoginDelegatedLoading());

    final failureOrData = await _loginDelegatedRequestByPhone(id);
    failureOrData.fold(
        (failure) => emit(LoginDelegatedError(failure.toString())),
        (data) => emit(LoginDelegatedRequested('phone_number')),
    );
}
```

Figura 20: Codi Cubit Request OTP code

La funció `sendLoginDelegatedRequestByPhone` gestiona el disparador inicial del flux. Com s'observa a la captura:

- **Inici de l'acció:** Immediatament després de ser invocada, la funció emet l'estat `LoginDelegatedLoading()`. Això permet a la UI mostrar visualment un indicador de càrrega, bloquejant interaccions duplicades.
- **Processament asíncron:** S'espera el resultat de la crida a la capa de dades. Mitjançant el mètode `fold` (propi de la programació funcional), es gestionen els dos camins possibles:
 - **Error (failure):** S'emet `LoginDelegatedError`, enviant el missatge d'error corresponent per ser mostrat a l'usuari.
 - **Èxit (data):** S'emet `LoginDelegatedRequested('phone_number')`, indicant que el codi s'ha enviat correctament i que el sistema està llest per a la següent fase.

```

Future<void> sendLoginDelegatedValidateByPhone(
    String id,
    String otpCode,
) async {
    emit(LoginDelegatedLoading());

    final failureOrData = await _loginDelegatedValidateByPhone(id, otpCode);
    failureOrData.fold(
        (failure) => emit(LoginDelegatedError(failure.toString())),
        (data) {
            if (data == null) {
                emit(LoginDelegatedOtpError('Error: Incorrect otp Code'));
            } else {
                emit(LoginDelegatedValidated(data));
            }
        },
    );
}

```

Figura 21: Codi Cubit Validate OTP code

La funció `sendLoginDelegatedValidateByPhone` tanca el cicle d'autenticació processant el codi introduït per l'usuari:

- **Gestió de la transició:** Igual que en el pas anterior, s'emet un estat de càrrega (Loading) per mantenir el feedback visual.
- **Validació lògica:** Després d'executar la validació, el codi avalua la resposta detalladament:
 - Si la resposta és nul·la o incorrecta, s'emet un estat específic d'error de codi: `LoginDelegatedOtpError('Error: Incorrect otp Code')`. Aquesta granularitat en els estats permet que la interfície s'enfoqui directament en el camp de text del codi sense reiniciar tot el formulari.
 - Si la validació és satisfactòria, s'emet `LoginDelegatedValidated(data)`, enviant les dades de l'usuari autenticat i permetent la navegació a la pantalla principal.

11.8.3. Definició de l'Arquitectura d'Estats (States)

```
part of 'login_delegated_cubit.dart';

abstract class LoginDelegatedState {}

class LoginDelegatedInitial extends LoginDelegatedState {}

class LoginDelegatedLoading extends LoginDelegatedState {}

class LoginDelegatedRequested extends LoginDelegatedState {
  final String phoneOrMail;

  LoginDelegatedRequested(this.phoneOrMail);
}

class LoginDelegatedValidated extends LoginDelegatedState {
  final LoginResponseEntity? result;

  LoginDelegatedValidated(this.result);
}

class LoginDelegatedError extends LoginDelegatedState {
  final String message;

  LoginDelegatedError(this.message);
}

class LoginDelegatedOtpError extends LoginDelegatedState {
  final String message;

  LoginDelegatedOtpError(this.message);
}
```

Figura 22: Codi Estats del Cubit

L'ecosistema d'estats es divideix segons les etapes del flux d'autenticació:

- **Estats de Transició i Càrrega:**
 - LoginDelegatedInitial: Estat base quan el bottom sheet s'obre per primera vegada.

- LoginDelegatedLoading: Estat transversal que s'emet durant les crides asíncrones al backend (com s'ha vist en les funcions `sendLoginDelegatedRequestByPhone` i `sendLoginDelegatedValidateByPhone`), utilitzat per bloquejar el botó d'acció i mostrar un indicador de progrés.
- **Estats de Flux i Dades:**
 - LoginDelegatedRequested: S'emet un cop el backend confirma l'enviament de l'OTP. Aquest estat és crucial ja que transporta la variable `phoneOrMail`, assegurant que la identitat de l'usuari persisteixi en la transició cap a la pantalla d'introducció del codi.
 - LoginDelegatedValidated: Representa l'èxit final del procés. Transporta un objecte `LoginResponseEntity?` result, que conté la informació de l'usuari i els tokens necessaris per a l'accés global a l'aplicació.
- **Gestió Granular d'Errors:** S'ha optat per una especialització d'errors per millorar l'experiència d'usuari (UX):
 - LoginDelegatedError: Estat genèric per a fallades de xarxa o de servidor durant qualsevol fase del procés.
 - LoginDelegatedOtpError: Estat específic per a codis de validació incorrectes. Aquesta distinció permet que la interfície mantingui l'usuari a la pantalla del codi OTP amb un missatge d'advertència localitzat, en lloc de reiniciar tot el flux de login.

11.8.4. Implementació de la Interfície Reactiva (Capa de Presentació)

```
66 @override
67 Widget build(BuildContext context) {
68   return BlocConsumer<LoginDelegatedCubit, LoginDelegatedState>(
69     listener: (_, state) {
70       if (state is LoginDelegatedValidated) {
71         _loginCubit.setApiTokenDelegated(state.result);
72         _loginCubit.resetState();
73         appRouter.pop();
74       }
75
76       if (state is LoginEntityApiUpdated) {
77         if (widget.loginDelegatedUser != null) {
78           widget.loginDelegatedUser!();
79         }
80       }
81     },
82
83     builder: (_, state) {
84       if (state is LoginDelegatedLoading) {
85         return const Center(child: CircularProgressIndicator());
86       }
87
88       if (state is LoginDelegatedOtpError) {
89         return OtpContentModal(
90           emailOrPhone: _emailOrPhone,
91           onOtpConfirmed: _onOtpConfirmed,
92           resend: _onResend,
93           goBack: _loginCubit.resetState,
94           isErrorMessage: true,
95         ); // OtpContentModal
96       }
97     }
98   );
99 }
```

Figura 23: Codi Bloc Part 1

```

96     }
97
98     if (state is LoginDelegatedRequested) {
99         return OtpContentModal(
100             emailOrPhone: _emailOrPhone,
101             onOtpConfirmed: _onOtpConfirmed,
102             resend: _onResend,
103             goBack: _loginCubit.resetState,
104             isErrorMessage: false,
105         ); // OtpContentModal
106     }
107
108     if (state is LoginDelegatedError) {
109         return Text(state.message);
110     }
111
112     return EmailOrPhoneModal(
113         onContinueWithPhone: _onContinueWithPhone,
114         onContinueWithMail: _onContinueWithMail,
115     ); // EmailOrPhoneModal
116 },
117 ); // BlocConsumer
118 }

```

Figura 24: Codi Bloc Part 2

Gestió d'esdeveniments i navegació (Listener): Com s'observa en el fragment de codi del listener, el sistema reacciona a canvis d'estat que no requereixen un canvi visual directe en el widget actual, sinó accions secundàries:

- **Validació reeixida:** Quan l'estat és `LoginDelegatedValidated`, s'executa el mètode `setApiTokenDelegated` per actualitzar la sessió global de l'usuari, es reinicia l'estat del Cubit i es tanca el bottom sheet mitjançant `appRouter.pop()`.
- **Sincronització de perfil:** L'ús de `LoginEntityApiUpdated` garanteix que, un cop autènticat, s'executin les funcions de callback (com `loginDelegatedUser`) per carregar la informació correcta de l'usuari des del backend.

Renderització dinàmica de la UI (Builder): El builder és l'encarregat de retornar el component visual adequat segons l'estat en què es troba la màquina d'estats del Cubit. Aquesta implementació permet una experiència d'usuari fluida sense canvis de pantalla bruscs:

- **Indicador de càrrega:** Si l'estat és `LoginDelegatedLoading`, es mostra un `CircularProgressIndicator` centrat, proporcionant el feedback necessari durant les crides asíncrones.
- **Pantalla d'introducció d'OTP:** Quan l'estat és `LoginDelegatedRequested` o `LoginDelegatedOtpError`, la interfície muta al widget `OtpContentModal`.
 - S'injecta la variable `isErrorMessage` com a `true` si l'estat és d'error de codi, permetent que el modal mostri l'advertència visual corresponent sense perdre la informació introduïda.
- **Pantalla d'identificació inicial:** Per defecte (estat inicial o errors genèrics), es mostra el `EmailOrPhoneModal`. Aquest component conté la lògica de detecció d'input que permet a l'usuari triar entre continuar amb telèfon o correu electrònic.

Conclusió tècnica: Aquesta estructura basada en el patró BLoC garanteix que la interfície sigui una **funció pura de l'estat**. El codi resultant és altament mantenible i escalable, ja que qualsevol nova funcionalitat o canvi en el flux d'autenticació només requereix la definició d'un nou estat i la seva gestió en aquest `BlocConsumer`.

12. Testing i Validació

Aquest capítol descriu les estratègies i tècniques utilitzades per validar que el sistema compleix els requisits funcionals i no funcionals definits en el capítol d'Especificació. L'objectiu principal del procés de testing no ha estat únicament detectar errors, sinó garantir la qualitat, robustesa i fiabilitat del sistema dins d'un entorn de producció real.

Atès que el projecte s'ha desenvolupat en el marc d'una aplicació ja existent i en ús per usuaris reals, el testing s'ha centrat principalment en proves d'integració, proves funcionals i validacions manuals guiades pels criteris d'acceptació definits.

12.1. Estratègia general de testing

L'estratègia de testing aplicada combina diferents tipus de proves:

- **Testing manual funcional**, basat en criteris d'acceptació.
- **Proves d'integració** entre front-end i back-end.
- **Validació de requisits no funcionals**, com rendiment, usabilitat i mantenibilitat.
- **Bones pràctiques de desenvolupament**, orientades a prevenir errors abans de la fase de testing.

No s'ha aplicat una estratègia de testing unitari exhaustiu degut a la naturalesa del projecte i al fet que moltes funcionalitats formen part d'un sistema ja consolidat. En canvi, s'ha prioritzat la validació del comportament real del sistema des del punt de vista de l'usuari final.

12.2. Validació dels requisits funcionals

La validació dels requisits funcionals s'ha dut a terme principalment mitjançant **tests manuals**, alineats amb els criteris d'acceptació definits durant la fase d'Especificació.

Per a cada funcionalitat implementada s'ha seguit el següent procés:

1. Execució de l'acció corresponent des de la interfície d'usuari.
2. Verificació del comportament esperat en el front-end.
3. Validació de la resposta del backend (codi HTTP i contingut del JSON).
4. Comprovació de la persistència correcta de les dades a la base de dades, quan escau.

Aquest procés s'ha repetit tant en entorns de desenvolupament com en entorns de producció.

12.2.1. Testing de la comunicació Front-end / Back-end

La comunicació entre el front-end i el back-end s'ha validat mitjançant:

- Proves directes dels endpoints utilitzant **Postman**.
- Simulació d'escenaris reals des de l'aplicació Flutter.
- Validació de casos d'error, com respostes incorrectes, errors de validació o problemes d'autenticació.

Aquestes proves han permès detectar errors d'integració abans que arribessin a producció.

12.2.2. Testing de funcionalitats clau

Algunes funcionalitats especialment crítiques, com la nova Home, el Delegated login o la funcionalitat Cintia, han estat sotmeses a un procés de validació més exhaustiu.

Aquest testing ha inclòs:

- Canvis d'estat de la interfície.
- Gestió de càrregues asíncrones.
- Comportament de la UI davant errors del backend.
- Resposta del sistema davant absència de dades o dades incompletes.

Aquest enfocament ha garantit una experiència d'usuari coherent i robusta.

12.3. Validació dels requisits no funcionals

Els requisits no funcionals definits mitjançant la plantilla VOLERE han estat validats a través de diferents mecanismes, segons la seva naturalesa.

12.3.1. Rendiment

El rendiment s'ha validat mitjançant:

- Observació dels temps de resposta dels endpoints.
- Anàlisi del comportament de la interfície en pantalles amb càrrega elevada de dades.
- Optimització de widgets Flutter per evitar reconstruccions innecessàries.

Els resultats han estat satisfactoris, mantenint temps de resposta adequats per a una aplicació mòbil en producció.

12.3.2. Usabilitat

La usabilitat s'ha validat mitjançant:

- Proves manuals d'ús continuat de l'aplicació.
- Validació de fluxos de navegació coherents.
- Adaptació de la interfície a diferents mides de pantalla.

Els canvis implementats a la Home han millorat significativament la claredat visual i la comprensió de la informació per part de l'usuari.

12.3.3. Mantenibilitat

La mantenibilitat s'ha validat indirectament mitjançant:

- L'aplicació estricta de patrons arquitectònics (Clean Architecture i Arquitectura Hexagonal).
- L'ús de linters i convencions de codi.
- Revisió de codi mitjançant Pull Requests.

Aquestes pràctiques faciliten l'evolució futura del projecte i la incorporació de nous desenvolupadors.

12.3.4. Seguretat

La seguretat s'ha validat mitjançant:

- Control d'accés als endpoints del backend.
- Validació de dades d'entrada.
- Gestió segura de tokens i credencials.

Tot i que la seguretat global del sistema depèn en gran part de l'arquitectura existent, les funcionalitats desenvolupades respecten els mecanismes establerts.

12.4. Bones pràctiques com a mecanisme de validació

Una part important de la validació del sistema s'ha dut a terme de manera preventiva, mitjançant bones pràctiques de desenvolupament:

- Ús de patrons de disseny reconeguts.
- Revisió de codi abans de cada merge.
- Validació automàtica de l'estil del codi mitjançant linters.
- Separació clara de responsabilitats entre capes.

Aquest enfocament ha reduït significativament l'aparició d'errors crítics durant les fases finals del projecte.

12.5. Conclusions del procés de validació

El procés de testing i validació ha permès confirmar que:

- Els requisits funcionals s'han implementat correctament.
- Els requisits no funcionals es compleixen dins dels marges establerts.
- El sistema és estable, usable i mantenible.

Això valida el projecte com una solució tècnica adequada dins del context per al qual ha estat desenvolupat.